

CSCE 753 COMPUTER VISION · MODULE 13 · LECTURE NOTES

Vision in Systems

Running continuously, on real hardware, over time.

MODULE OUTCOMES

- Reason about the latency budget of a perception system.
- Choose where computation happens.
- Explain what degrades over time and how to detect it.
- Design a perception system that knows when it is unreliable.
- State the course's closing position.

1 Latency

A LATE ANSWER IS A WRONG ANSWER

For any system that acts on the world, perception carries a deadline. A detection that takes 200 milliseconds on a moving platform describes where objects *were*, not where they are — and at 10 m/s that is two metres of error that no accuracy improvement addresses.

So the correct metric is accuracy *at the deadline*, and a faster, less accurate model is frequently the better choice. Streaming perception metrics exist to measure exactly this — they evaluate the most recent available output against the current world state, which penalises latency and inaccuracy on the same scale.

And the budget is end to end: capture and exposure, transfer to the accelerator, preprocessing, inference, postprocessing, and the action itself. **Inference is often not the largest term, and optimising it first is a common and expensive mistake** — quantising a model to save 10 ms while 25 ms goes to CPU-side image resizing.

So measure the whole pipeline before optimising any part of it. CSCE 735 Module 02's profiling discipline and CSCE 678 Module 06's tail-latency lesson both apply directly here, and the second is the one that causes production incidents: a pipeline sized for the median fails at the 99th percentile, which on a 30 Hz camera happens eighteen times a minute.

Stage	Typical cost	What helps
Capture and exposure	8–33 ms	A higher frame rate, a shorter exposure — and note that exposure time is a latency term, which people forget.

Stage	Typical cost	What helps
Transfer to the accelerator	1–10 ms	Pinned memory, and avoiding GPU-to-CPU-to-GPU round trips , which are the usual hidden cost here.
Preprocessing	1–20 ms	Do resizing and normalisation on the GPU. Frequently the single largest forgotten cost , because it looks like bookkeeping rather than computation.
Inference	5–100 ms	Quantisation, pruning, distillation, or simply a smaller model (CSCE 636 Module 09).
Postprocessing (NMS)	1–30 ms	Batched GPU non-maximum suppression, and capping the candidate count before it — an unbounded candidate list makes this term unbounded too.
Tail latency	2–5× the median	Budget for p99, not the mean — CSCE 678 Module 06. Garbage collection, thermal throttling, and contention all land here.

2 Where computation happens

- **On device. Lowest latency, no network dependency, and no image ever leaves the user's hardware.** Constrained by memory, by power, and by thermal budget — and the thermal constraint is real: sustained inference throttles, so a benchmark measured cold overstates sustained performance.
- **On a local edge machine.** More compute, a few milliseconds of local network, and images stay on the premises — which is frequently the deciding factor in regulated settings rather than the performance.
- **In the cloud.** Effectively unlimited compute, 50–500 ms of round trip including the unpredictable parts, **and every frame is transmitted and may be retained.**
- **Hybrid is the usual answer:** a small on-device model handles the common case and escalates to a larger one only when it is uncertain. **The cascade needs a usable uncertainty estimate to route on** — and Module 12 §3 established that you do not get one for free, so the calibration work is a prerequisite for the architecture rather than a refinement of it.
- **And camera data is personal data. Where it goes is a design decision with consequences well beyond latency** — retention, access, subject rights, and the irreversibility of a leak. **On-device inference is a privacy property, not only a performance one**, and it is worth choosing deliberately rather than arriving at by default.

3 Degradation over time

WHAT DECAYS

CALIBRATION	thermal drift, mechanical shock, lens cleaning, remounting (Module 02 section 4)
THE SCENE	new signage, repainted walls, a camera nudged by 2 degrees, seasonal foliage, a new product line
THE LENS	dust, scratches, a smeared protective cover
THE CAMERA	sensor ageing, and an auto-exposure firmware update that changed the response
THE LABELS	what the operators consider a positive drifts over months -- concept shift (Module 12 section 1)

HOW TO NOTICE WITHOUT GROUND TRUTH

track the INPUT distribution: mean brightness, contrast, sharpness, detected feature count per frame. A shift here PRECEDES an accuracy drop and NEEDS NO LABELS.

track the OUTPUT distribution: detections per hour, the confidence histogram, the class mix. A change with no known external cause is the alarm.

track REPROJECTION ERROR continuously if you do geometry -- it is a free and direct calibration monitor.

reserve a FIXED set of reference inputs and re-score them on a schedule, so you have a controlled comparison.

and SAMPLE outputs for human review on a schedule, not only when something already looks wrong.

A SYSTEM WITH NO MONITORING IS NOT DEPLOYED.
IT IS ABANDONED IN PRODUCTION.

Monitoring the input distribution needs no labels and catches most degradation early, which makes it the highest-value thing to build and the first thing to build. **It is also the one piece of this that transfers unchanged to every other learned system** — CSCE 633 Module 13 made the same argument, and in vision it is easier, because the input statistics of an image are cheap to compute and genuinely informative.

4 Knowing when not to answer

BUILD THE REFUSAL IN

A system that answers everything confidently is less useful than one that abstains when uncertain, because **the downstream consumer can handle 'unknown' and cannot handle 'wrong'** — an unknown triggers a fallback, a human, or a retry, while a confident error propagates.

Geometry gives you honest signals, for free. RANSAC inlier counts (Module 03 §3), reprojection error (Module 05 §2), estimate conditioning (Module 01 §2), left–right stereo consistency (Module 06 §4), track closure error (Module 07 §4). **Every one of these is already computed and usually discarded. Use them.**

Learned components do not provide this by default (Module 12 §3), so it must be added: an explicit out-of-distribution detector, a calibrated confidence with a threshold chosen from the cost of each error type, or an ensemble disagreement measure.

And make the abstention visible in the output type. A depth map with a validity mask (Module 06 §4) **is the model for this: the measurement, together with where it is not a measurement. An interface that cannot express 'I do not know' forces every component downstream to guess on its behalf,** and that is where the hardest failures in perception systems come from.

WHERE THIS COURSE LEAVES YOU

You can calibrate a camera and verify the calibration independently; detect, describe, and match features robustly; recover relative pose and triangulate; run and diagnose a structure from motion pipeline; compute dense depth and know where it is fabricated; and estimate motion while knowing what the estimate assumes. And you can diagnose a failed reconstruction from its symptom, which is the skill that distinguishes someone who has used these tools from someone who has read about them.

You can train and honestly evaluate a learned component, choose a transfer strategy from your data volume, pick a metric from the use rather than the leaderboard, detect a shortcut, and **state what the system can be trusted to do and under what conditions.**

And you understand Module 11 in both directions — which is the thing this track exists to produce. **The forward model is what you invert; the inversion is ill-posed; and both facts are load-bearing in every method in this course.** Module 01 claimed that and Module 11 demonstrated it.

The closing rule is the program's, unchanged across nineteen courses: state what you measured, state what you assumed, and never claim more than you established. In computer vision it means stating the capture conditions — the camera, the lighting, the sites, the season — **because that is what the system quietly learned alongside the task you intended to teach it.**

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Li, Wang & Ramanan — Towards Streaming Perception (free)	The §1 metric. Evaluates perception against the world's current state rather than the frame's, which is the right formulation.
NVIDIA TensorRT and ONNX Runtime documentation (free)	The practical tools for §1's inference row , including quantisation and the profiling output that shows where the time goes.
Sculley et al. — Hidden Technical Debt in Machine Learning Systems (free)	The §3 argument , and the clearest short statement of why monitoring is not optional.
Szeliski — Computer Vision, chapter 14 and the conclusions (free PDF)	A survey of applications and open problems , and a reasonable place to choose what to study next.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Profile your own perception pipeline end to end** and report each stage's cost.
- 2 **Report the p50, p95, and p99 latency** and the ratio between them.
- 3 **Move preprocessing to the GPU** and report the change.
- 4 **Measure sustained throughput over twenty minutes** and report the thermal throttling.
- 5 **Implement a two-stage cascade** with a small model routing to a large one, and report the accuracy-latency trade.
- 6 **Log input-distribution statistics** for your system over a week and plot them.
- 7 **Deliberately smear the lens** and confirm the statistics move before accuracy does.
- 8 **Track reprojection error continuously** and plot it against camera temperature.
- 9 **Add an abstention path** to your system using a geometric signal, and report how often it fires.
- 10 **Project 2 is now due.** Submit the trained component, the separately-collected test results, the per-subgroup and corrupted numbers, the failure gallery, the latency measurement, and the scoped claim.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Why is a late answer a wrong answer, and what metric follows?
- 2 Name the six stages of a latency budget and the two most often forgotten.
- 3 Why budget for p99 rather than the median?
- 4 Compare device, edge, and cloud placement on three axes.

- 5 Why is on-device inference a privacy property?
- 6 Name five things that degrade, and five ways to detect degradation without labels.
- 7 Why is input-distribution monitoring the highest-value monitor?
- 8 Why is abstention more useful than confident answering?
- 9 Name five geometric signals you already compute and could abstain on.